Contents

1. Data Visualization and its importance 2
2. Libraries used for data visualization 2
3. Matplotlib Libraray 2
4. Seaborn Libraries 3
5. Scatter plot 6
6. Line Plot 7
7. Bar Plot 9
8. Histogram 11
9. Box plot 12
10. Pair plot 15

Module - 4 Data Visualization on using Matplotlib and Seaborn libraries

1. **Data Visualization and its importance**

   Data visualization is the representation of data through use of common graphics, such as charts, plots, infographics, and even animations. Data visualizations help us understand the underlying structure within a dataset or explore the relationships among variables. The importance of Data visualization is – analyzing complex data, identifying patterns, and extracting valuable insights. There are various data visualization tools as Microsoft Power BI, Tableau, Qlik Sense, Klipfolio. Looker, Zoho Analytics, Domo etc.

2. **Libraries used for data visualization**

   Python provides various libraries that come with different features for visualizing data. All these libraries come with different features and can support various types of graphs.

   - Matplotlib
   - Seaborn
   - Bokeh
   - Plotly

3. **Matplotlib Libraray**

   Matplotlib is an easy-to-use, low-level data visualization library that is built on NumPy arrays. It consists of various plots like scatter plot, line plot, histogram, etc. Matplotlib provides a lot of flexibility. Matplotlib is a powerful data visualization tool that allows you to view large amounts of data in comprehensible forms. It was launched by the John Hunter in 2002. We can use the below command to install it: **pip install Matplotlib**

   Advantages of Matplotlib library:

   1. **Matplotlib provides a simple way to access large amounts of data**

      Developers can create accurate plots based on huge amounts of data. When analyzing these data plots, good developers will make it easier to see patterns and trends in the data sets.

   2. **It is flexible and supports various forms of data representation**

      Matplotlib supports data representation in bar charts, graphs, scattered plots, and other forms of visualization.

   3. **It is easy to navigate**

      The Matplotlib platform isn't too complex. Hence, both beginners and advanced developers can apply their programming skills to the platform, producing professional results. Matplotlib also has subplots that further facilitate the creation and comparison of data sets.

   4. **It ensures accessibility by providing high-quality images**

The main goal of Matplotlib is to provide a way to access and display data, its plots and images must be of high quality. To meet this requirement, Matplotlib provides high-quality images in various formats, such as PDF, PGF, and PNG.

5. **It is a powerful tool with numerous applications**

   Matplotlib data-visualization qualities can be used in various forms, such as Python scripts, shells, web application servers, and Jupyter notebooks.

6. **It is open-source**

   An open-source platform requires no paid license. Because Matplotlib is free to use, you save the extra cost you usually incur when producing data visualizations.

7. **It can run on different platforms**

   Matplotlib is platform-independent. This means it can run smoothly no matter what platform you use. Whether your developers use Windows, Mac OS, or Linux, you can expect high-quality results.

8. **It makes data analysis easier**

   Due to its numerous features, plot styles, and high-quality results, Matplotlib makes data analysis easier and more efficient. It also helps save the time and resources you would have spent analyzing large datasets.

Disadvantages of Matplotlib library

Matplotlib is excellent for drumming up charts and graphics but it may not be the best fit for time series data. There are few disadvantages as listed below:

1. library is not the best for dealing with time series data since one needs to import all these helper classes for the year, month, week, day formatters, shared a user on a forum.
2. Library is very low-level, which means that one needs to write more code to get the visualization as opposed to Tableau where you can achieve that in a few clicks.

4. **Seaborn Libraries**

Seaborn is one of an amazing library for visualization of the graphical statistical plotting in Python. Seaborn provides many color palettes and defaults beautiful styles to make the creation of many statistical plots in Python more attractive.

Seaborn library aims to make a more attractive visualization of the central part of understanding and exploring data. It is built on the core of the Matplotlib library and also provides dataset-oriented APIs.

Different categories of plot in Seaborn: Plots are basically used for visualizing the relationship between variables. Those variables can be either completely numerical or a category like a group, class, or division. Seaborn divides the plot into the below categories –

A. **Relational plots:** This plot is used to understand the relation between two variables.
B. **Categorical plots:** This plot deals with categorical variables and how they can be visualized. There are some key categories plots are like Barplot, Countplot , Boxplot , Violinplot, Stripplot, Swarmplot , Factorplot.
C. **Distribution plots:** This plot is used for examining univariate and bivariate distributions. It has various categories like : Displot, Joinplot ,
D. **Regression plots:** The regression plots in Seaborn are primarily intended to add a visual guide that helps to emphasize patterns in a dataset during exploratory data analyses. It has also two categories like Simple linear plot, displaying multiple plots.
E. **Matrix plots:** A matrix plot is an array of scatterplots.
F. **Multi-plot grids:** It is a useful approach to draw multiple instances of the same plot on different subsets of the dataset.

Advantages of Seaborn Libraries:

1. By using the Seaborn library, we can easily represent our data on a plot.
2. This library is used to visualize our data. No need to take care of the internal details; we just have to pass our data set or data inside the **relplot ()** function, and it will calculate and place the value accordingly.
3. We can switch to any other representation of data using the 'kind' property inside it.
4. It creates an interactive and informative plot to representation our data; also, this is easy for the user to understand and visualize the records on the application.
5. It uses static aggregation for plot generation in python.

Disadvantages of Seaborn Libraries:

1. **Complexity for Beginners:** Seaborn's extensive range of plot types and customization options can be overwhelming for beginners. Users who are new to data visualization or programming might find it challenging to grasp and utilize all of its features effectively.
2. **Limited Customization**: While Seaborn makes it easy to create aesthetically pleasing plots with minimal code, it might be limiting for users who require highly customized and unique visualizations. Advanced customizations might require working directly with the underlying Matplotlib functions.
3. **Learning Curve:** While Seaborn simplifies many aspects of data visualization, there's still a learning curve, especially if users want to take full advantage of its capabilities. Understanding the relationships between different plot functions, using various parameters, and navigating the documentation can take some time.

4. **Performance:** Seaborn is built on top of Matplotlib, which can sometimes result in performance bottlenecks when dealing with very large datasets. While Seaborn tries to optimize its performance, it might not be as efficient as some other libraries for handling big data visualization tasks.
5. **Integration with Other Libraries:** While Seaborn works well with Pandas DataFrames, it might not seamlessly integrate with other data manipulation libraries or tools, which could be a drawback for users who rely on different data processing workflows.
6. **Few Built-in Statistical Tests:** Seaborn provides some basic statistical functionalities like correlation matrix visualizations and simple linear regression plots. However, it might lack the breadth and depth of statistical tests and visualizations that specialized statistical packages like R's ggplot2 offer.

Difference between Matplotlib and Seaborn Library

Matplotlib	Seaborn
Matplotlib creates simple graphs, including bar graphs, histograms, pie charts, scatter plots, lines, and other visual representations of data.	There are numerous patterns and graphs for data visualization in Seaborn. It employs engaging themes, and it helps in the integration of all data into a single plot. Additionally, it offers data distribution.
It utilizes syntax that is relatively complicated and extensive. Example: Matplotlib.pyplot.bar (x-axis, y-axis) is the syntax for a bar graph.	It has relatively simple syntax, making it simpler to learn and comprehend. Example: seaborn.barplot(x axis, y-axis) syntax for a bar graph.
We can open and work with many figures at once. You can close the current figure using the syntax:matplotlib.pyplot.close(). Close all the figures using this syntax: matplotlib.pyplot.close("all")	Seaborn sets the time for the creation of each figure. However, it may lead to (OOM) memory issues.
Matplotlib is a Python graphics package for data visualization and integrates nicely with Numpy and Pandas. Similar capabilities and syntax are available in Pyplot as in MATLAB, and users of MATLAB can readily understand it.	Seaborn is more comfortable with Pandas data frames. It utilizes simple sets of techniques to produce lovely images in Python.
Matplotlib is highly customized and robust.	With the help of its default themes, Seaborn prevents overlapping plots.
Matplotlib plots various graphs using Pandas and Numpy.	Seaborn is the extended version of Matplotlib, which uses Matplotlib, Numpy, and Pandas to plot graphs.

Seaborn is licensed under the GNU GPL.	Matplotlib is licensed under the Python Software Foundation License.

5. **Scatter plot**

A scatter plot is a graph in which the values of two variables are plotted along two axes, the pattern of the resulting points revealing any correlation present.

Scatterplot can be used with several semantic. They can plot two-dimensional graphics that can be enhanced by mapping up to three additional variables while using the semantics of hue, size, and style parameters.

Syntax: seaborn.scatterplot(x=None, y=None, hue=None, style=None, size=None, data=None, palette=None, hue_order=None, hue_norm=None, sizes=None, size_order=None, size_norm=None, markers=True, style_order=None, x_bins=None, y_bins=None, units=None, estimator=None, ci=95, n_boot=1000, alpha='auto', x_jitter=None, y_jitter=None, legend='brief', ax=None, **kwargs)

Parameters:

X, y: Input data variables that should be numeric.

Data: Data frame where each column is a variable and each row is an observation.

Size: Grouping variable that will produce points with different sizes.

Style: Grouping variable that will produce points with different markers.

Palette: Grouping variable that will produce points with different markers.

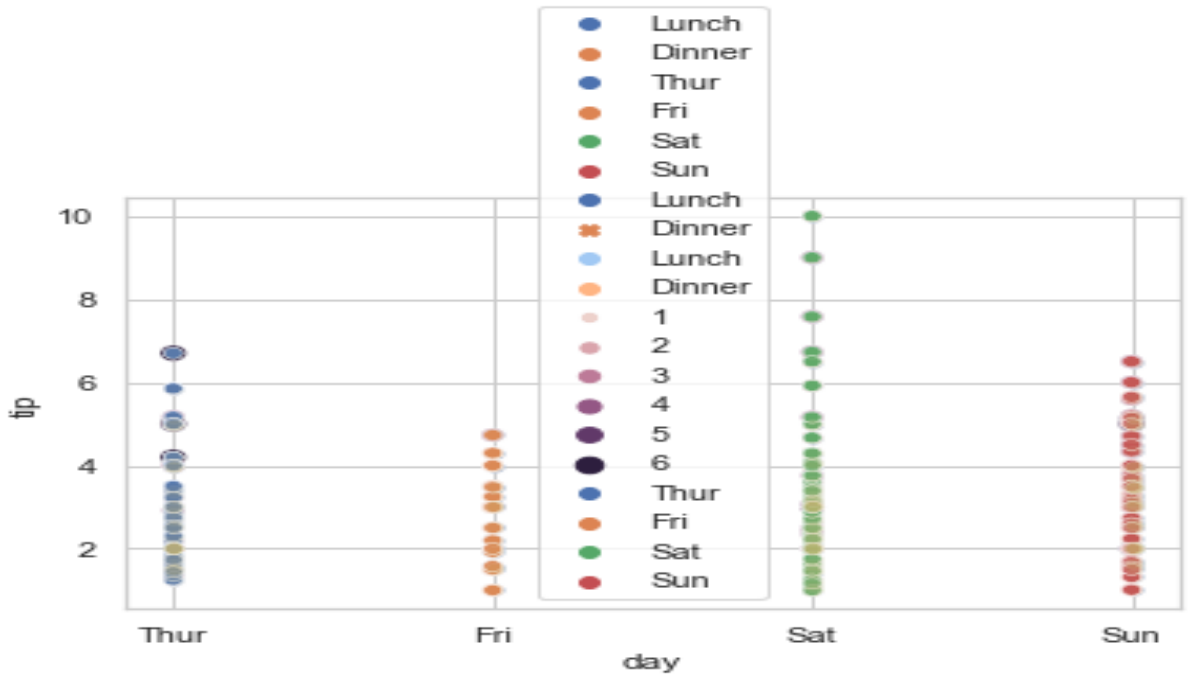
Markers: Object determining how to draw the markers for different levels.

Alpha: Proportional opacity of the points.

Returns: This method returns the Axes object with the plot drawn onto it.

```
# importing required modules
import pandas as pd
import numpy as np
import seaborn
# loading the dataset
dataset = seaborn.load_dataset('tips')
# heading of the dataset
#print(dataset.head(20))
seaborn.set(style='whitegrid')
tip = seaborn.load_dataset('tips')
```

```
seaborn.scatterplot(x='day', y='tip', data=tip)
#Adding the marker attributes
seaborn.scatterplot(x='day', y='tip', data= tip, marker = '+')
#Adding the hue attributes.
seaborn.scatterplot(x='day', y='tip', data=tip, hue='time')
seaborn.scatterplot(x='day', y='tip', data=tip, hue='day')
#Adding the style attributes.
seaborn.scatterplot(x='day', y='tip', data=tip, hue="time", style="time")
#Adding the palette attributes.
seaborn.scatterplot(x='day', y='tip', data=tip, hue='time', palette='pastel')
#Adding size attributes.
seaborn.scatterplot(x='day', y='tip', data=tip ,hue='size', size = "size")
#Adding legend attributes.
seaborn.scatterplot(x='day', y='tip', data=tip, hue='day',sizes=(30, 200), legend='brief')
#Adding alpha attributes.
seaborn.scatterplot(x='day', y='tip', data=tip, alpha = 0.1)
```



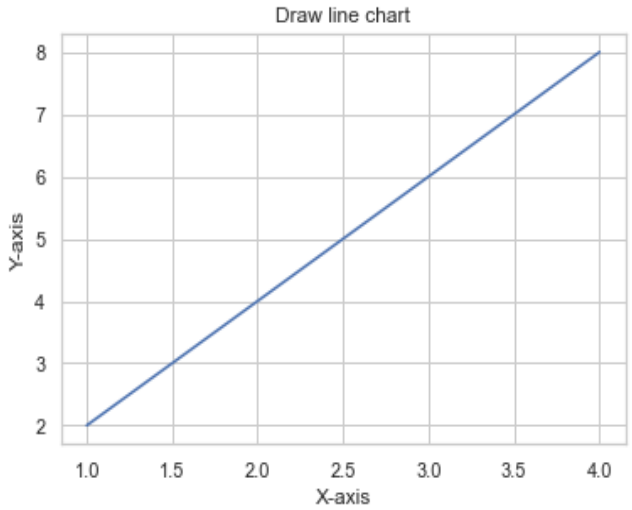
6. Line Plot

Line charts are used to represent the relation between two data X and Y on a different axis. We can draw line chart as given below:

- **Single line graph**

```
Import matplotlib.pyplot as plt
```

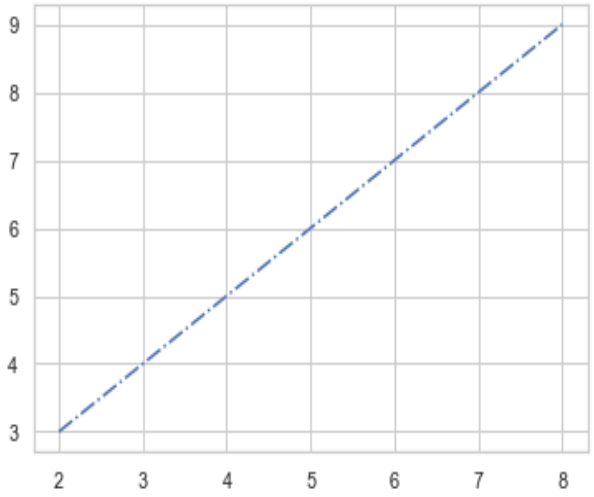
```
Import numpy as np
# Define X and Y variable data
x = np.array ([1, 2, 3, 4])
y = x*2
plt.plot(x, y)
plt.xlabel("X-axis")  # add X-axis label
plt.ylabel("Y-axis")  # add Y-axis label
plt.title("Draw Line chart")  # add title
plt.show()
```



❖ **Multiple charts in a line graph**

We can display more than one chart in the same container by using pyplot.figure() function.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1, 2, 3, 4])
y = x*2
plt.plot(x, y)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("My Graph")
plt.show()  # show first chart
# The figure() function helps in creating a
# new figure that can hold a new chart in it.
plt.figure()
x1 = [2, 4, 6, 8]
y1 = [3, 5, 7, 9]
plt.plot(x1, y1, '-.')
# Show another chart with '-' dotted line
plt.show()
```

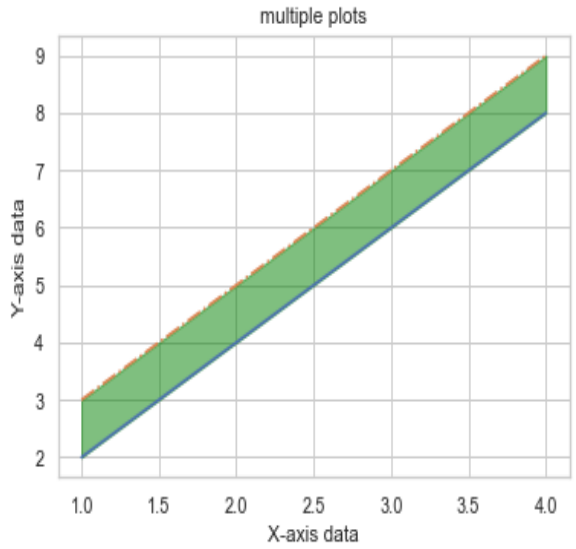


❖ **Fill the area between two plots**

Using the pyplot.fill_between () function we can fill in the region between two line plots in the same graph.

```
Import matplotlib.pyplot as plt
Import numpy as np
```

```
x = np.array ([1, 2, 3, 4])
y = x*2
plt.plot(x, y)
x1 = [2, 4, 6, 8]
y1 = [3, 5, 7, 9]
plt.plot(x, y1, '-.')
plt.xlabel("X-axis data")
plt.ylabel("Y-axis data")
plt.title('multiple plots')
plt.fill_between(x, y, y1, color='green', alpha=0.5)
plt.show()
```



7. Bar Plot

A bar plot or bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent. The bar plots can be plotted horizontally or vertically.

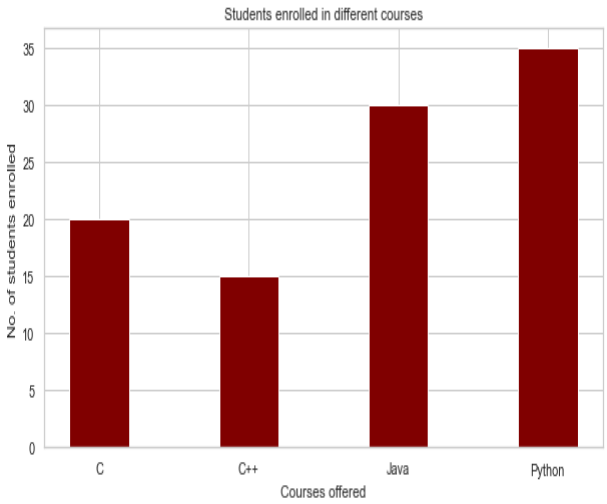
A bar chart describes the comparisons between the discrete categories. One of the axis of the plot represents the specific categories being compared, while the other axis represents the measured values corresponding to those categories.

❖ **Creating a bar plot:**

The matplotlib API in Python provides the **bar ( )** function which can be used in MATLAB style use or as an object-oriented API. The syntax of the bar() function to be used with the axes is as follows:-

Syntax: plt.bar(x, height, width, bottom, align)

```
import numpy as np
import matplotlib.pyplot as plt
# creating the dataset
data = {'C':20, 'C++':15, 'Java':30, 'Python':35}
courses = list(data.keys())
values = list(data.values())
    fig = plt.figure(figsize = (10, 5))
     # creating the bar plot
    plt.bar(courses, values, color ='maroon',width = 0.4)
     plt.xlabel("Courses offered")
    plt.ylabel("No. of students enrolled")
```

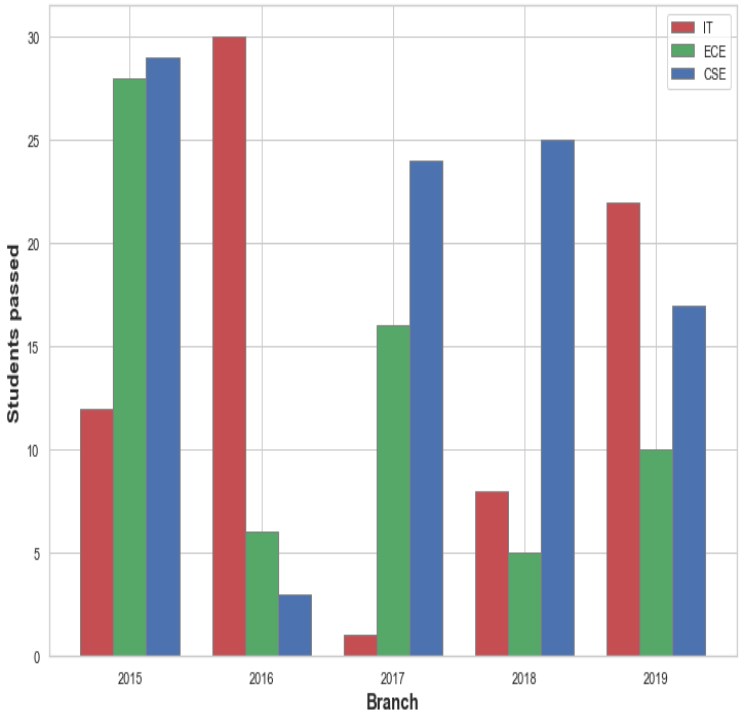


```
plt.title("Students enrolled in different courses")
plt.show()
```

❖ **Multiple bar plots**

Multiple bar plots are used when comparison among the data set is to be done when one variable is changing. We can easily convert it as a stacked area bar chart, where each subgroup is displayed by one on top of the others. It can be plotted by varying the thickness and position of the bars**.**

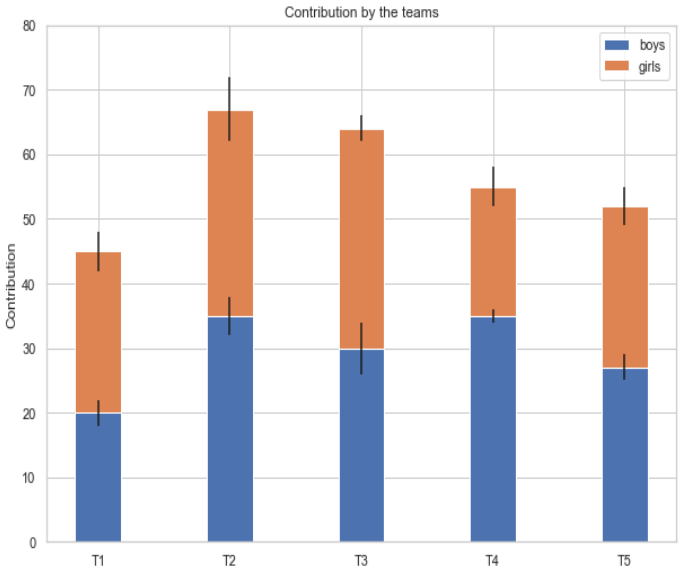
```
import numpy as np
import matplotlib.pyplot as plt
 # set width of bar
barWidth = 0.25
fig = plt.subplots(figsize =(12, 8))
# set height of bar
IT = [12, 30, 1, 8, 22]
ECE = [28, 6, 16, 5, 10]
CSE = [29, 3, 24, 25, 17]
# Set position of bar on X axis
br1 = np.arange(len(IT))
br2 = [x + barWidth for x in br1]
br3 = [x + barWidth for x in br2]
# Make the plot
plt.bar(br1, IT, color ='r', width = barWidth,
edgecolor ='grey', label ='IT')
plt.bar(br2, ECE, color ='g', width = barWidth,
edgecolor ='grey', label ='ECE')
plt.bar(br3, CSE, color ='b', width = barWidth,
edgecolor ='grey', label ='CSE')
# Adding Xticks
plt.xlabel('Branch', fontweight ='bold', fontsize = 15)
plt.ylabel('Students passed', fontweight ='bold', fontsize = 15)
plt.xticks([r + barWidth for r in range(len(IT))],['2015', '2016', '2017', '2018', '2019'])
plt.legend()
plt.show()
```



❖ **Stacked bar plot**

Stacked bar plots represent different groups on top of one another. The height of the bar depends on the resulting height of the combination of the results of the groups. It goes from the bottom to the value instead of going from zero to value.

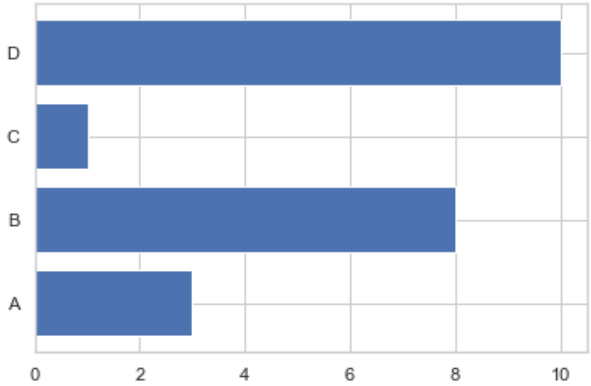
```
import numpy as np
import matplotlib.pyplot as plt
N = 5
boys = (20, 35, 30, 35, 27)
girls = (25, 32, 34, 20, 25)
boyStd = (2, 3, 4, 1, 2)
girlStd = (3, 5, 2, 3, 3)
ind = np.arange(N)
width = 0.35
fig = plt.subplots(figsize =(10, 7))
p1 = plt.bar(ind, boys, width, yerr = boyStd)
p2 = plt.bar(ind, girls, width,  bottom = boys, yerr = girlStd)
plt.ylabel('Contribution')
plt.title('Contribution by the teams')
plt.xticks(ind, ('T1', 'T2', 'T3', 'T4', 'T5'))
plt.yticks(np.arange(0, 81, 10))
plt.legend((p1[0], p2[0]), ('boys', 'girls'))
plt.show()
```



❖ **Horizontal Bars**

If we want the bars to be displayed horizontally instead of vertically, use the barh() function.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
plt.barh(x, y)
plt.show()
```



8. Histogram

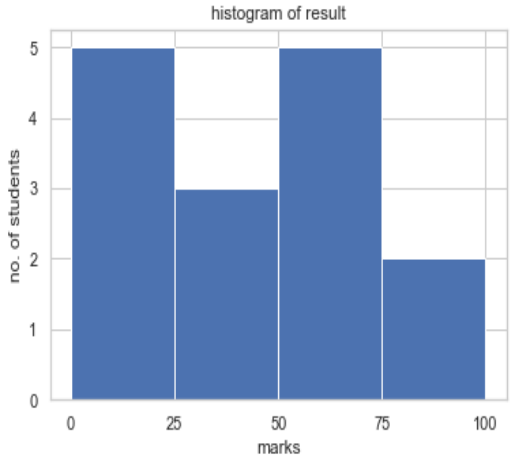
A histogram is an accurate representation of the distribution of numerical data. It is an estimate of the probability distribution of a continuous variable. It is a kind of bar graph.

To construct a histogram, we follow these steps –

- ❖ Bin the range of values.
- ❖ Divide the entire range of values into a series of intervals.
- ❖ Count how many values fall into each interval.

The **matplotlib.pyplot.hist ()** function plots a histogram. It computes and draws the histogram of x.

```
from matplotlib import pyplot as plt
import numpy as np
fig,ax = plt.subplots(1,1)
a = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
ax.hist(a, bins = [0,25,50,75,100])
ax.set_title("histogram of result")
ax.set_xticks([0,25,50,75,100])
ax.set_xlabel('marks')
ax.set_ylabel('no. of students')
plt.show()
```



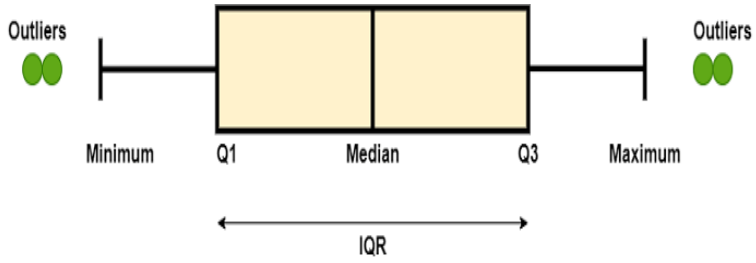
9. Box plot

A Box Plot is also known as Whisker plot is created to display the summary of the set of data values having properties like minimum, first quartile, median, third quartile and maximum.

A Box plot is a way to visualize the distribution of the data by using a box and some vertical lines. It is known as the whisker plot.

The data can be distributed between five key ranges, which are as follows:

- ❖ Minimum: Q1-1.5*IQR
- ❖ 1st quartile (Q1): 25th percentile
- ❖ Median:50th percentile
- ❖ 3rd quartile(Q3):75th percentile
- ❖ Maximum: Q3+1.5*IQR



Creating Box Plot

The **matplotlib.pyplot** module of **Matplotlib** library provides boxplot () function with the help of which we can create box plots.

Syntax: matplotlib.pyplot.boxplot(data, notch=None, vert=None, patch_artist=None, widths=None)

```
import matplotlib.pyplot as plt
import numpy as np
 # Creating dataset
np.random.seed(10)
```

```
data = np.random.normal(100, 20, 200)
fig = plt.figure(figsize =(10, 7))
# Creating plot
plt.boxplot(data)
# show plot
plt.show()
```

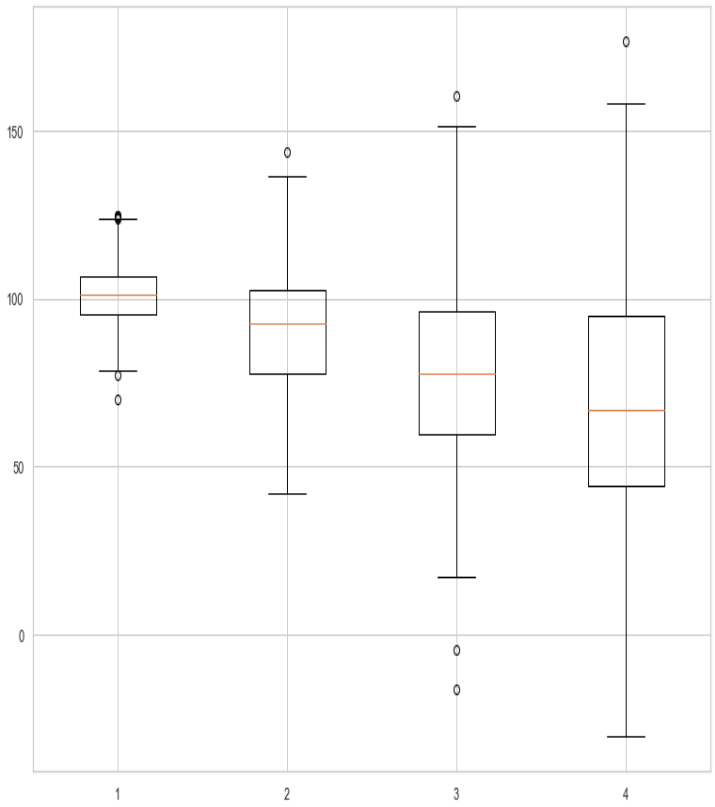


❖ **Customizing Box Plot**

The matplotlib.pyplot.boxplot() provides endless customization possibilities to the box plot. The notch = True attribute creates the notch format to the box plot, patch_artist = True fills the boxplot with colors, we can set different colors to different boxes.

The vert = 0 attribute creates horizontal box plot. labels takes same dimensions as the number data sets.

```
import matplotlib.pyplot as plt
import numpy as np
 # creating dataset
np.random.seed(10)
 data_1 = np.random.normal(100, 10, 200)
data_2 = np.random.normal(90, 20, 200)
data_3 = np.random.normal(80, 30, 200)
data_4 = np.random.normal(70, 40, 200)
data = [data_1, data_2, data_3, data_4]
 fig = plt.figure(figsize =(10, 7))
# Creating axes instance
ax = fig.add_axes([0, 0, 1, 1])
 # Creating plot
bp = ax.boxplot(data)
# show plot
plt.show()
```

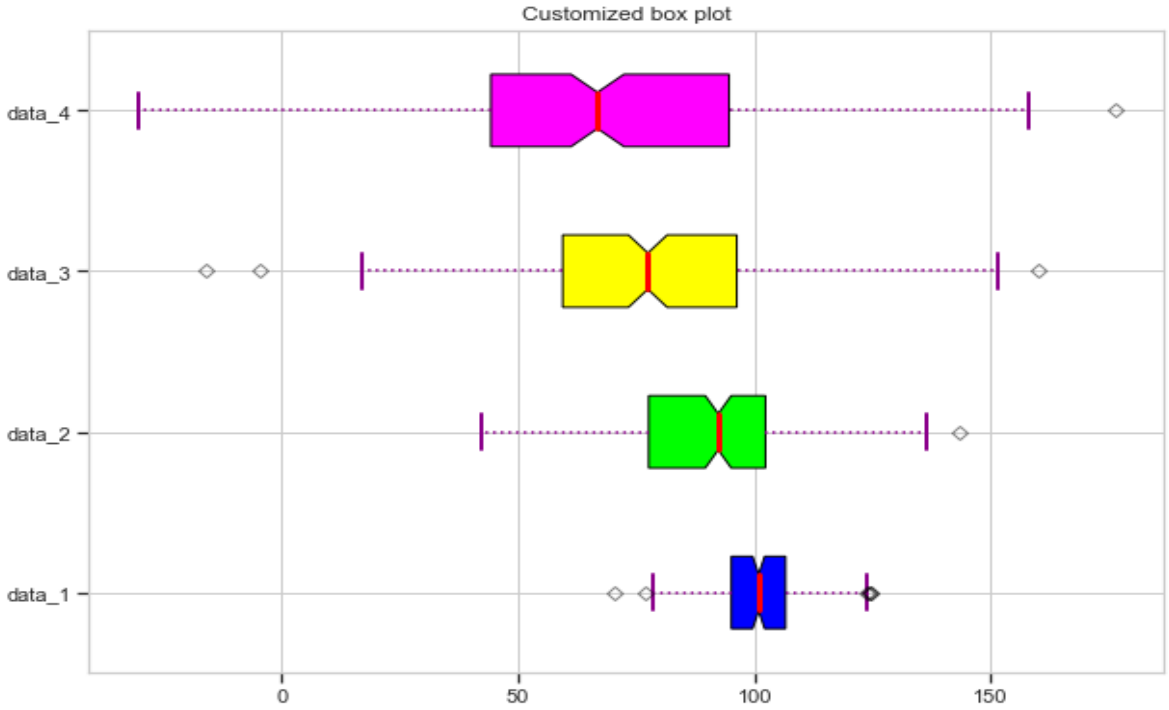


❖ **Some of the customizations:**

```
import matplotlib.pyplot as plt
import numpy as np
 # Creating dataset
np.random.seed(10)
```

```
data_1 = np.random.normal(100, 10, 200)
data_2 = np.random.normal(90, 20, 200)
data_3 = np.random.normal(80, 30, 200)
data_4 = np.random.normal(70, 40, 200)
data = [data_1, data_2, data_3, data_4]
fig = plt.figure(figsize =(10, 7))
ax = fig.add_subplot(111)
# Creating axes instance
bp = ax.boxplot(data, patch_artist = True,  notch ='True', vert = 0)
colors = ['#0000FF', '#00FF00', '#FFFF00', '#FF00FF']
for patch, color in zip(bp['boxes'], colors): patch.set_facecolor(color)
 # changing color and linewidth of
# whiskers
for whisker in bp['whiskers']:
      whisker.set(color ='#8B008B',  linewidth = 1.5,  linestyle =":")
     # changing color and linewidth of
     # caps
     for cap in bp['caps']:
      cap.set(color ='#8B008B',  linewidth = 2)
     # changing color and linewidth of
     # medians
     for median in bp['medians']:
        median.set(color ='red', linewidth = 3)
     # changing style of fliers
     for flier in bp['fliers']:
       flier.set(marker ='D', color ='#e7298a', alpha = 0.5)
     # x-axis labels
     ax.set_yticklabels(['data_1', 'data_2',  'data_3', 'data_4'])
     # Adding title
     plt.title("Customized box plot")
     # Removing top axes and right axes
     # ticks
     ax.get_xaxis().tick_bottom()
     ax.get_yaxis().tick_left()
```

```
# show plot
plt.show()
```



10. Pair plot

The **Seaborn.pairplot ( )** method is used to plot pairwise relationships in a dataset. Each numeric variable in the data will be spread over the y-axes across a single row and the x-axes across a single column by default, according to the axes grid created by this function.

Syntax: Following is the syntax of the seaborn.pairplot () method –

seaborn.pairplot(data, *, hue=None, hue_order=None, palette=None, vars=None, x_vars=None, y_vars=None, kind='scatter', diag_kind='auto', markers=None, height=2.5, aspect=1, corner=False, dropna=False, plot_kws=None, diag_kws=None, grid_kws=None, size=None)

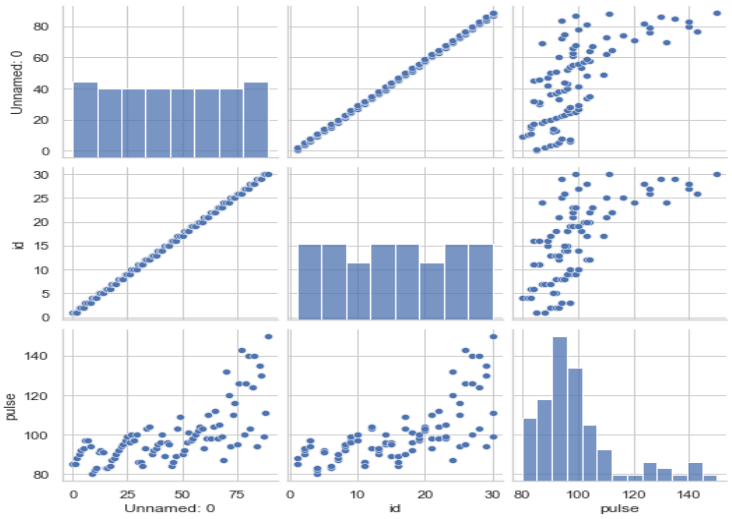
Parameters

S.No	Name and Description
1	**Data:** Takes dataframe where each column is a variable and each row is an observation.
2	**Hue:** variables that specify portions of the data that will be displayed on certain grid facets. To regulate the order of levels for this variable, refer to the var order parameters.
3	**Kind:** Takes values from{‘scatter’, ‘kde’, ‘hist’, ‘reg’}and the kind of plot to make is determined.
4	**Diag_kind: Takes** values from {‘auto’, ‘hist’, ‘kde’, None} and the kind of plot for the diagonal subplots is determined if used. If ‘auto’, choose based on whether or not hue is used.

5	**Height:** Takes a scalar value and determines the height of the facet.
6	**Aspect:** Takes scalar value and Aspect ratio of each facet, so that aspect * height gives the width of each facet in inches.
7	**Corner:** Takes Boolean value and If True, don't add axes to the upper (off-diagonal) triangle of the grid, making this a "corner" plot.
8	**hue_order:** Takes lists as input and Order for the levels of the faceting variables is determined by this order.

Creation of pair plot:

```
import seaborn as sns
import matplotlib.pyplot as plt
exercise =sns.load_dataset("exercise")
exercise.head()
sns.pairplot(exercise)
plt.show()
```



For Example:

```
import seaborn as sns
import matplotlib.pyplot as plt
exercise =sns.load_dataset("exercise")
exercise.head()
g = sns.pairplot(exercise, diag_kind="kde")
g.map_lower(sns.kdeplot, levels=4, color=".8")
plt.show()
```

